

Sponsored by :



INNOVITUS 1.0

Produced by

iTriangle
Innovate • Optimise • Deliver

TEAM NAME : CACHE COWS
TEAM LEADER: THUSHITHA R
TRACK 2: WEB DEV / SAAS DOMAIN
MAIL ID: THUSHITHAR.CY23@RVCE.EDU.IN



PROBLEM STATEMENT



PS 3: Build a "near-zero friction" capture system that acts as a version control layer for musical ideas.

Problem:

- Creative ideas are often lost because capturing them requires interrupting the flow with manual recording tools like DAWs.
- Existing systems store music as files, not evolving ideas, making it hard to revisit, connect, or build on fragments.
- As a result, there is no structured way to preserve, search, and develop spontaneous musical inspiration.

Let’s meet Arjun, an independant musician, who works with DAWs such as FL Studio or Bandlab.

Situation	Pain Point	What Arjun Needs
Playing a good melody casually	Ideas get lost due to delay in opening DAW and recording	A system that continuously buffers audio and lets him instantly capture the last few seconds without breaking his flow
Has many saved clips and voice notes	Unorganized, hard to search or recall ideas	A system that automatically tags and organizes ideas so he can search them based on meaning, not file names
Revisiting old ideas	Cannot track variations or build on ideas without losing previous versions	A system that allows him to branch, evolve, and combine ideas while preserving the original versions



Thus, we propose **FlowState**, an easy to use solution that captures audio retrospectively, extracts meaning from it, and organizes ideas as an evolving graph



SOLUTION



A “GitHub” for musicians that is designed to capture, understand, and evolve musical ideas
Our solution is structured as 4 phases:

Ambient Capture Layer

- Continuously records audio into a rolling buffer (30–60s)
- Captures the last few seconds instantly on trigger.

Semantic Indexing Layer

- Extracts audio features like BPM and frequency patterns
- Generates tags such as mood and instrument
- Makes clips searchable by meaning instead of file names

Idea Versioning Layer

- Stores each clip as a node in a graph structure
- Supports variations and merging of ideas
- Preserves all versions without overwriting

Retrieval & Interaction Layer

- Enables search using natural language queries
- Displays clips as organized, tagged units
- We use simple graph traversal (**DFS**) to explore, reconstruct, and visualize idea evolution.

What Arjun sees:

- Arjun casually hums or plays guitar
- He doesn't open any DAW
- When he likes something, he presses Capture

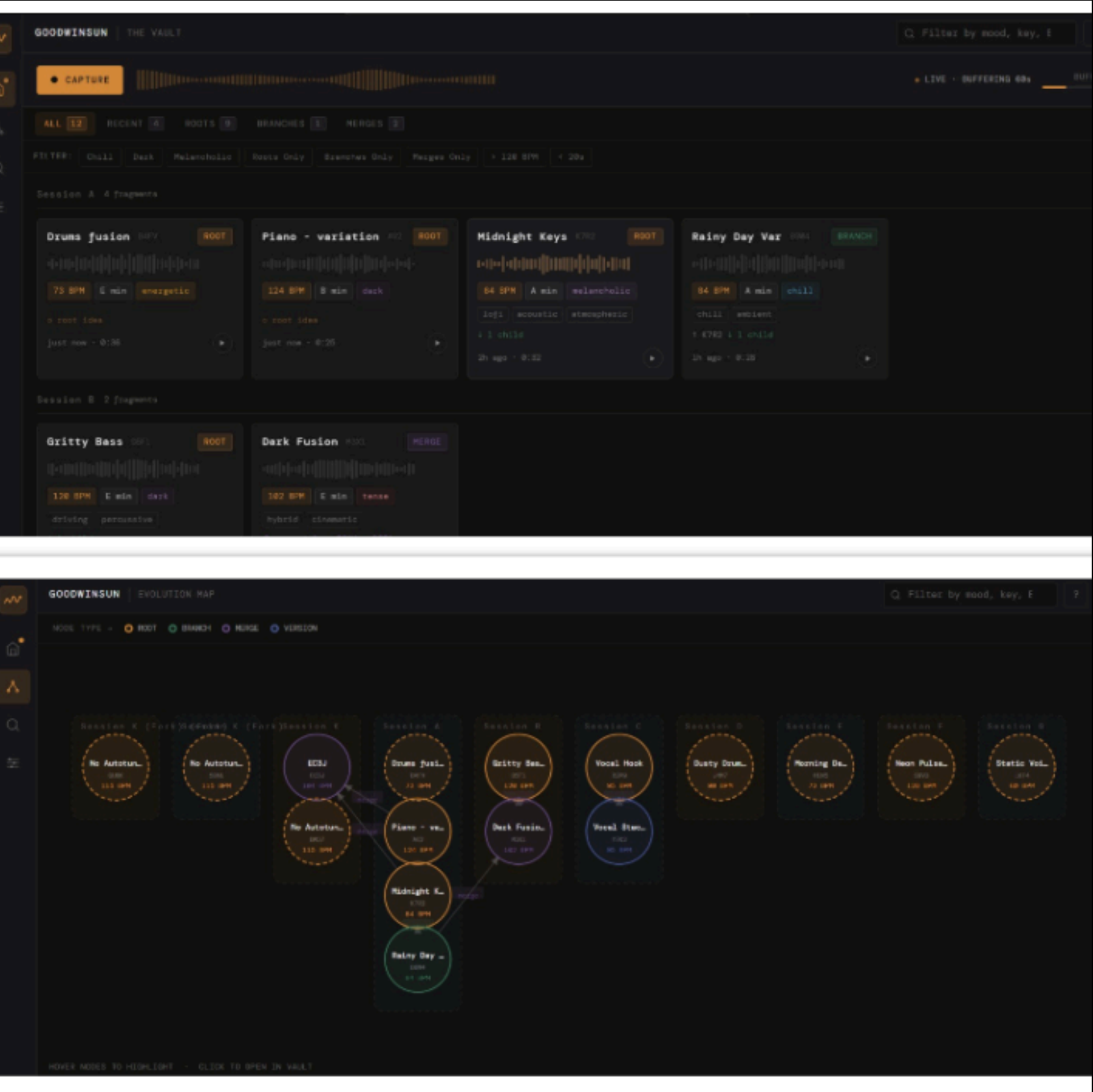
- Right after capture,
- Arjun sees:
 - BPM
 - Tags like chill, guitar
- He didn't label anything manually

- Arjun opens a clip
- He:
 - continues it → creates B
 - tries a different version → creates C

- He searches for “slow guitar idea”
- Finds the clip instantly
- He opens the graph and sees how idea evolved and plays full sequence



SOLUTION (UI Screen)

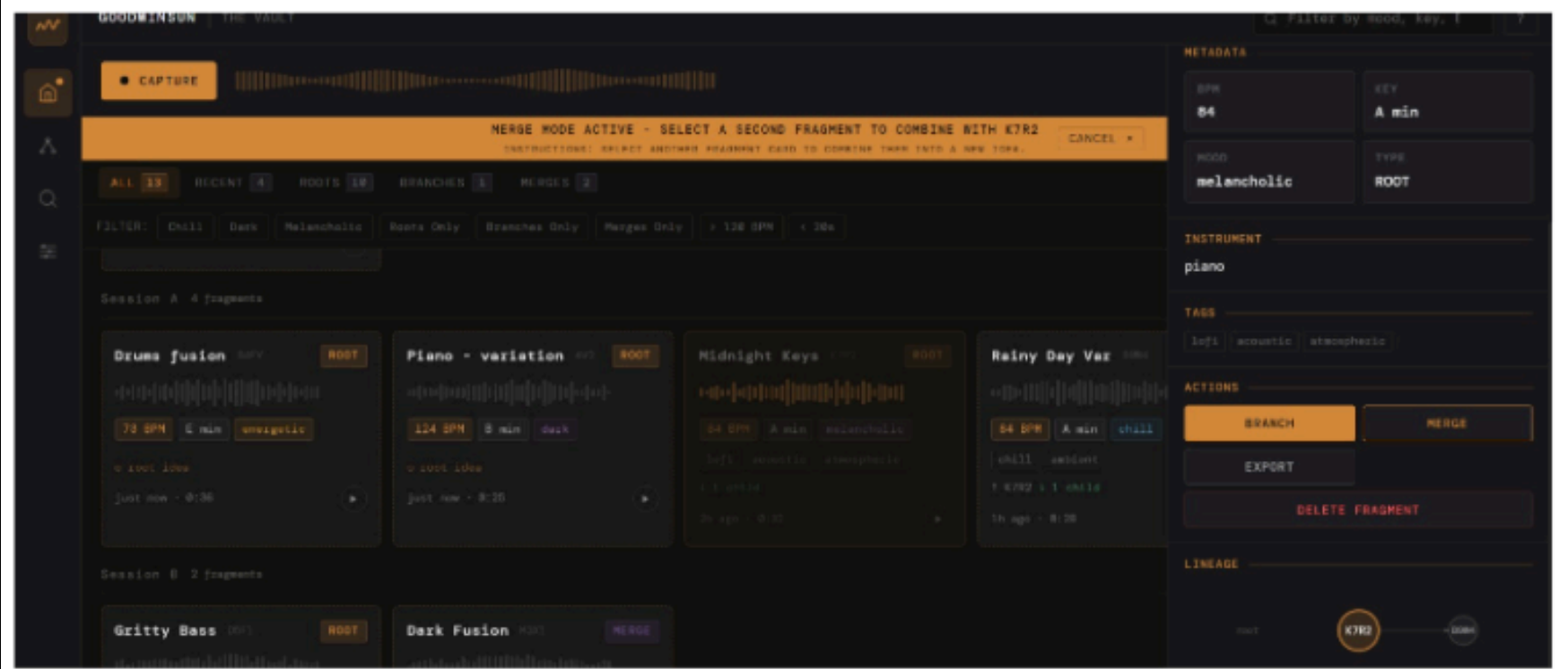
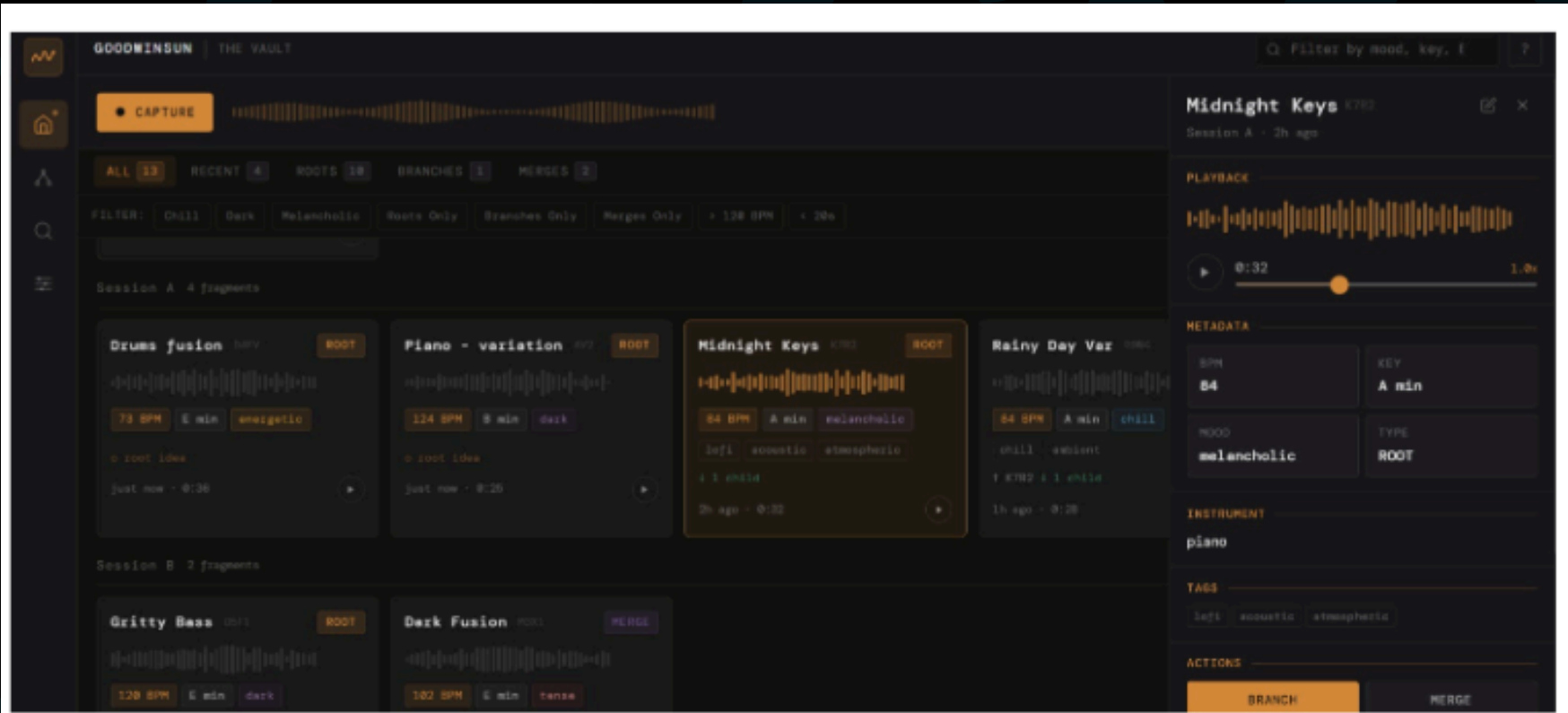


1. Intelligent Idea Capture & Organization
GOODWINSUN's Vault replaces traditional folders with a session-based, card-driven system that surfaces your musical ideas intuitively. Each fragment is displayed as a card with key metadata, BPM, key, and mood allowing for instant understanding without opening files. Ideas are structured as ROOT, BRANCH, or MERGE, making their creative relationships and evolution clear. A persistent CAPTURE bar continuously records in the background, ensuring no moment of inspiration is ever missed. Powerful filtering by mood, tempo, tags, enables quick and effortless retrieval

2. Visual Creative Evolution (Evolution Map)
The Evolution Map transforms your musical journey into a directed acyclic graph, where each node represents a fragment and each connection reflects a creative relationshipbranch, fork, merge, or version. Color-coded by type (ROOT, BRANCH, MERGE, VERSION), it provides a clear, visual understanding of how ideas evolve over time. With draggable, connectable nodes, you can actively shape and reorganize your creative flow. It functions as a visual “commit history” for music, offering a level of insight and control not found in traditional production workflows.



SOLUTION (UI Screen)



1. Merge — Creative Combination
Merging brings multiple ideas together into a single, unified fragment. It enables you to combine elements from different sessions or concepts such as rhythms and melodies into something new while preserving the originals. In the Evolution Map, merged nodes link back to all parent fragments, maintaining full history and attribution.

2. Fork — Independent Reinvention
Forking creates a complete copy of a fragment that breaks away from its original lineage. Unlike branching, which stays within the same creative thread, a fork establishes a new, parallel path often in a different session. This is ideal when an idea needs to be reimaged without constraints, allowing for a fresh direction while still retaining access to the original source.



KEY POINTS & FEATURES



SYSTEM FLOW

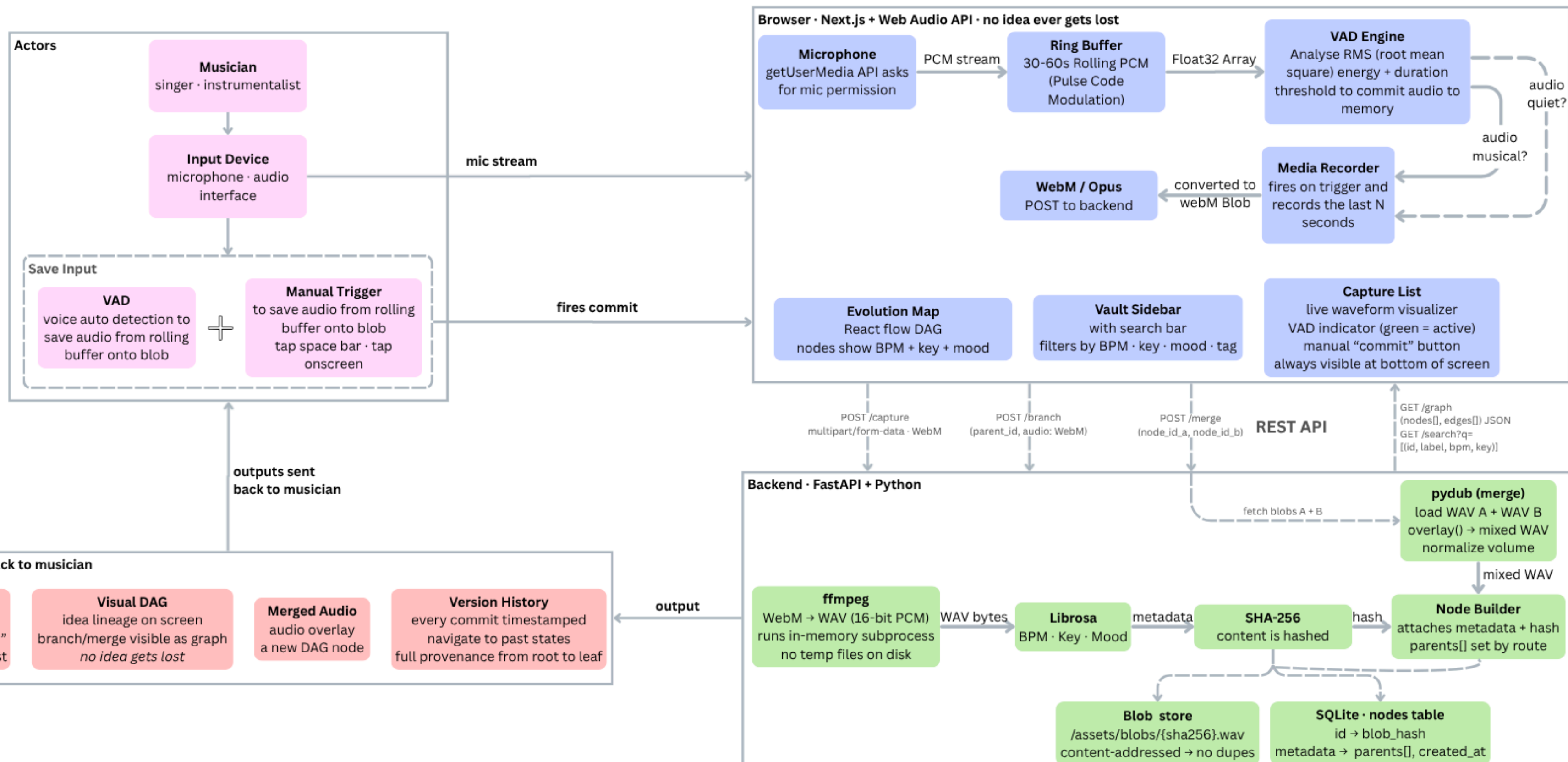


FEATURES

Ambient (Always-On) Capture	Retrospective Recording	AI-Powered Semantic Indexing
Constant rolling buffer (30–60s) Captures ideas after they happen No need to open a DAW or press record • Impact: removes friction → preserves flow state	Constant rolling buffer (30–60s) Captures ideas after they happen No need to open a DAW or press record • Impact: removes friction → preserves flow state	Extracts: BPM, Key, Mood Converts audio → searchable metadata • Impact: You can search ideas by meaning, not filenames
Idea-Based Storage (Not Files)		Graph-Based Organization
Parent → child relationships Supports: continuation (linear evolution), branching (variations), merging (combining ideas) Impact: Tracks how creativity evolves over time		Ideas stored as a network, not directories Impact: Organization emerges naturally from relationships
Semantic Search		Evolution Map (Visual Graph)
Query like: “chill guitar riff” Returns relevant clips using metadata/ embeddings Eliminates need to remember names or folders		Interactive graph showing idea lineage Tracks: sessions, branches, merges Impact: Makes creativity visible and explorable



ARCHITECTURE

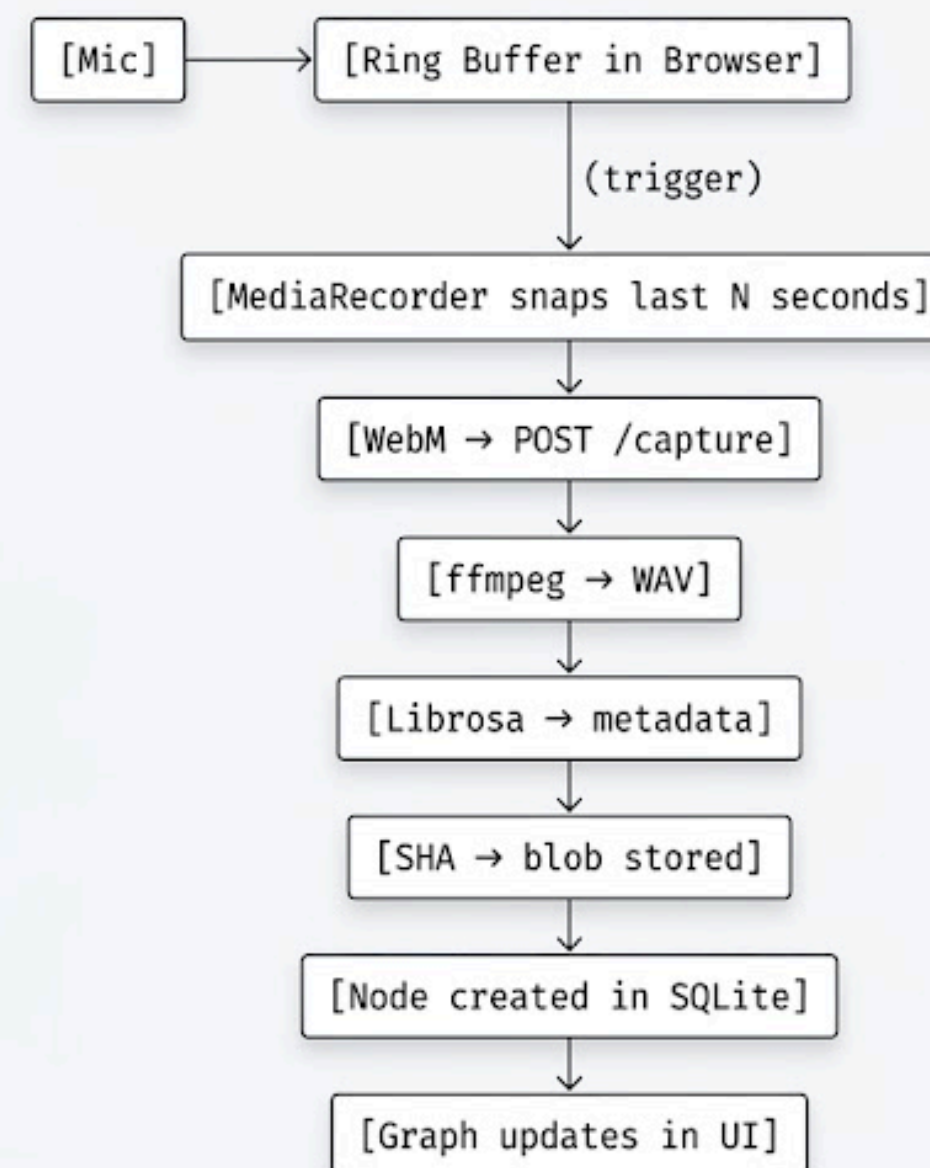




ARCHITECTURE

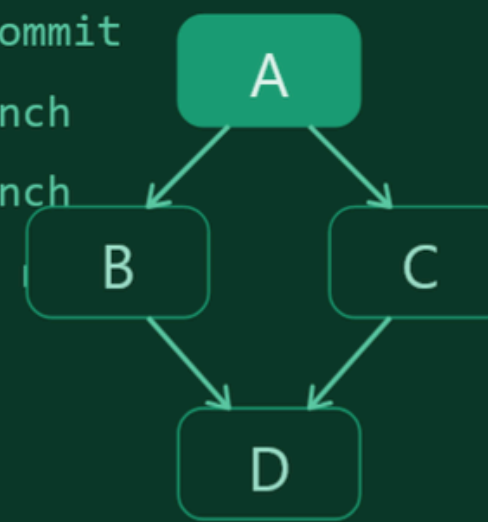
- The musician plays something — VAD or manual trigger fires, MediaRecorder slices the last 60s from the browser-side ring buffer and POSTs it to the backend as WebM.
- Backend converts to WAV, runs Librosa for metadata, SHA-256 hashes the audio, stores the file, and writes a node to SQLite with a parents[] field that encodes the DAG.
- Frontend reads the graph back, renders it as an interactive React Flow diagram — every capture, branch, and merge visible as a connected node you can click, search, and navigate.

Flow of Data/File Conversions



DAG · idea versioning
encoded entirely in parents[]
no graph DB needed

```
A parents:[] root commit  
B parents:["A"] branch  
C parents:["A"] branch  
D parents:["B","C"]
```



TECH STACK & TOOLS REQUIRED



Layer	Technology	Purpose
Frontend framework	Next.js	UI, routing, SSR
Audio capture	Web Audio API + MediaRecorder	Ring buffer, VAD, WebM encoding
DAG visualisation	React Flow	Interactive node graph
Backend framework	FastAPI	REST API, async endpoints
Audio conversion	ffmpeg	WebM → WAV
Semantic Audio analysis	Librosa, Gemini API	BPM, key, mood extraction
Hashing	Python hashlib (SHA-256)	Content-addressed blob storage
Database	SQLite	Node records, DAG edges via parents[]
Blob storage	Local filesystem	/blobs/{sha256}.wav

TECHNICAL SPECIFICATION



Ambient Capture

- Ring buffer: 60s circular Float32Array.
- MediaRecorder output: WebM/Opus
- Backend conversion: ffmpeg WebM → 16-bit PCM WAV
- Merge: pydub overlay at matching sample rates

Semantic Indexing using Librosa

- BPM detection
- Extract spectral features:
 - Rule-based tagging (mood, instrument)
 - Mood: spectral centroid + RMS energy → heuristic (bright/dark, energetic/calm)

Idea Versioning/Storage

- Blob naming: SHA-256 hex digest of raw WAV bytes → {64-char-hash}.wav
- SQLite node schema: id UUID, blob_hash TEXT, label TEXT, bpm REAL, key TEXT, mood TEXT, parents TEXT (JSON), created_at ISO8601
- DAG traversal: recursive CTE query on parents[] for lineage lookup

Frontend

- React Flow canvas: nodes draggable, zoomable, pannable
- Node card: waveform thumbnail (Web Audio OfflineAudioContext), BPM, key, mood badge
- Search: client-side filter on cached graph + server fallback for full-text



FUTURE SCOPE & CONCLUSION



Future Scope:

Smarter Audio Intelligence

- Improve accuracy of:
 - key detection
 - instrument recognition
 - mood classification

Advanced Versioning & Editing

- Develop a smart merge feature, that looks at tempo and key alignment while linking nodes
- Allow for partial editing of clips along with full edits

DAW Integration

- Allow direct export to DAWs such as FL Studio, Bandlab for further development of ideas
- Make FlowState available as a plugin within DAWs for enhanced production.

Collaboration and Cross Device Functionality

- Create cloud based vault so that FlowState is accesible across devices
- Allow users to share their idea graphs and create multi user graphs and branches



Thus,
We aim to build a system that accomplishes the following:

Uses a continuous circular audio buffer to enable instant retrospective capture of ideas using VAD (voice audio detection) and filtering noise.

Applies real-time audio feature extraction (BPM, spectral features) to generate searchable metadata for semantic search of clips.

Stores clips as nodes in a Directed Acyclic Graph (DAG) supporting continuation, branching, and merging of ideas to finally view evolution of ideas